

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



MACHINE LEARNING (CO3117)

ASSIGNMENT
MACHINE LEARNING

Supervisor: **TRƯỜNG VĨNH LÂN**

Student's ID	Full name	Email
2353081	Võ Nhật Tân	<code>tan.vonhatcse@hcmut.edu.vn</code>
2353318	Huỳnh Nguyễn Quốc Việt	<code>viet.huynh2102@hcmut.edu.vn</code>
2352703	Trần Bảo Phúc Long	<code>long.tranbaophuc@hcmut.edu.vn</code>
2352752	Nguyễn Mạnh Tri Minh	<code>minh.nguyenmanh@hcmut.edu.vn</code>
2353059	Lương Tấn Tài	<code>tai.luongbku@hcmut.edu.vn</code>

HO CHI MINH CITY, 5/2026

Table of Contents

1	Introduction	3
2	Exploratory Data Analysis (EDA)	4
2.1	Dataset Description	4
2.2	Class Distribution	4
2.3	Visual Inspection of Samples	5
2.4	Quantitative Feature Extraction	5
2.4.1	RGB Channel Distributions	6
2.4.2	Morphological Feature Distributions by Class	6
2.4.3	Image Quality Analysis	7
2.5	Feature Correlation Analysis	7
3	Preprocessing and Feature Extraction	9
3.1	Preprocess	9
3.1.1	Spatial Normalization	9
3.1.2	Tensor Conversion	9
3.1.3	Type Casting and Scaling	9
3.1.4	Statistical Normalization	9
3.2	Feature Extraction Architecture	10
3.2.1	VGG-16	10
3.2.2	ResNet18	12
3.2.3	Vision Transformer	13
4	Machine Learning Training	16
4.1	Algorithmic Selection and Implementation Strategies	16
4.2	Hardware Acceleration and Backend Routing	17
4.3	Hyperparameter Tuning Protocol	17
4.4	Evaluation Metrics	18
4.5	Results and Comparison	18
4.5.1	Overall Performance by Feature Representation	18
4.5.2	Evaluation of Traditional Classifiers	19
4.5.3	Computational Efficiency and Hardware Acceleration	19
5	Deep Learning Training	20
5.1	Transfer Learning Strategy	20
5.2	Backbone Architectures	21
5.2.1	ResNet18	22
5.2.2	Vgg16	23
5.2.3	Vit_b_16	23
5.3	Custom Classification Head	24
5.4	Training and Validation Performance	25
5.5	Results Comparison against Traditional ML	28
6	Conclusion	31



6.1	Summary of Findings	31
6.2	Limitations and Future Work	31

1 Introduction

In the agricultural sector, the accurate identification and classification of crop varieties are critical for ensuring seed purity, optimizing yield, and maintaining market value. Historically, the classification of rice varieties has relied on manual visual inspection by agricultural experts—a process that is inherently time-consuming, subjective, and susceptible to human error. However, the rapid advancement of computer vision and machine learning has introduced scalable, highly accurate automated alternatives that are transforming agricultural practices.

Image classification presents unique computational challenges due to the high dimensionality of pixel data and the subtle inter-class variances found in granular biological subjects like rice. Resolving these challenges requires a rigorous machine learning pipeline, beginning with comprehensive Exploratory Data Analysis (EDA) and preprocessing to understand dataset distributions and image characteristics. Traditionally, classifying such data relies heavily on robust manual feature extraction combined with standard classification algorithms like Support Vector Machines (SVM) or Random Forests. Conversely, modern deep learning paradigms utilize architectures such as Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs) to automatically learn complex, hierarchical feature representations directly from raw image matrices. While these deep learning approaches offer superior feature representation, they often demand substantially higher computational resources and GPU memory.

The core objective of this study is to systematically develop, analyze, and compare a complete visual recognition pipeline for rice classification. Specifically, this project evaluates the performance trade-offs between traditional feature extraction methods and modern deep learning-based feature embeddings when used as inputs for standard machine learning classifiers. By meticulously analyzing the accuracy gains against computational overhead across different configurations, this research aims to provide practical insights into designing efficient, reliable automated systems for the agricultural industry.

2 Exploratory Data Analysis (EDA)

2.1 Dataset Description

The dataset used in this work is the *An Image Dataset of Rice Varieties*, publicly available on Mendeley Data [1] and authored by Mohammad Shorif Uddin (version 3, published December 2023). The collection contains low-resolution photographs of 20 rice varieties that are native to different regions of Bangladesh. Samples were collected from both rural farming areas and local marketplaces to ensure variety in appearance and lighting conditions. All images were captured with an iPhone 11 camera at a $5\times$ optical zoom between October 18 and November 29, 2023.

The dataset is distributed in two variants:

- **Original images:** 4,730 JPG files, each sized 853×853 pixels, totalling approximately 268 MB (254 MB compressed).
- **Augmented images:** 23,650 JPG files, each sized 512×512 pixels, generated by applying rotations (90° left, 90° right, 180°) and horizontal flips to the originals.

Both variants are organised into 20 sub-directories, one per rice variety: *Subol Lota*, *Bashmoti* (*Deshi*), *Ganjiya*, *Shampakatar*, *Sugandhi Katarivog*, *BR-28*, *BR-29*, *Paijam*, *Bashful*, *Lal Aush*, *BR-Jirashail*, *Gutisharna*, *Red Cargo* (*Birui*), *Najirshail*, *Pahari Birui*, *Polao* (*Katari*), *Polao* (*Chinigura*), *Amon*, *Shorna-5*, and *Lal Binni*.

Subset used in this analysis. All EDA steps described below are performed on the **Original** image set only, which provides unmodified visual information and avoids any statistical artefacts introduced by augmentation.

2.2 Class Distribution

The first step is to count the number of images available for each rice variety. Figure 1 shows the per-class sample counts across all 20 classes.

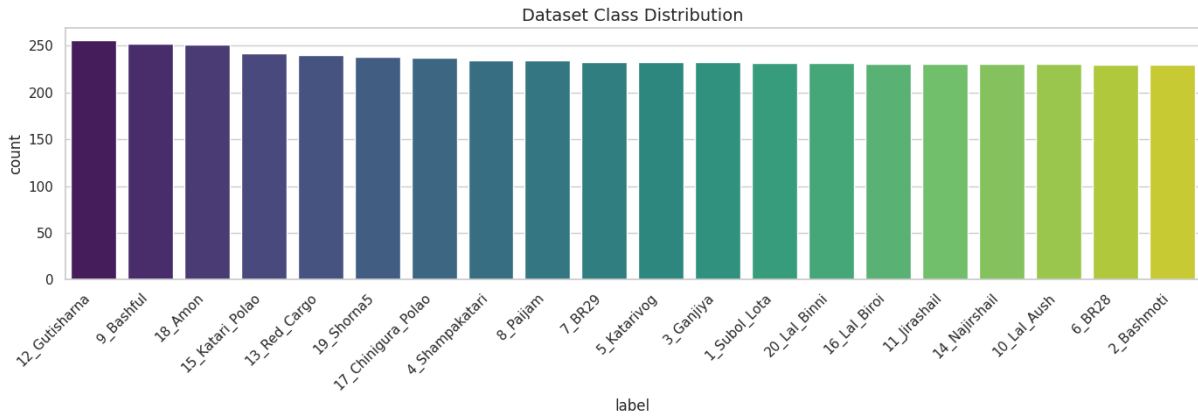


Figure 1: Per-class sample count across 20 rice varieties. Classes are sorted in descending order of count.

The dataset is relatively balanced; most classes contain a similar number of original images, with the total reaching 4,730 samples across 20 categories. This even distribution means that no single rice variety dominates the dataset, which reduces the risk of class-imbalance bias during model training.

2.3 Visual Inspection of Samples

To develop an intuitive understanding of the data, one representative image from each class is displayed after resizing to the target resolution of 224×224 pixels. Figure 2 presents the resulting grid.

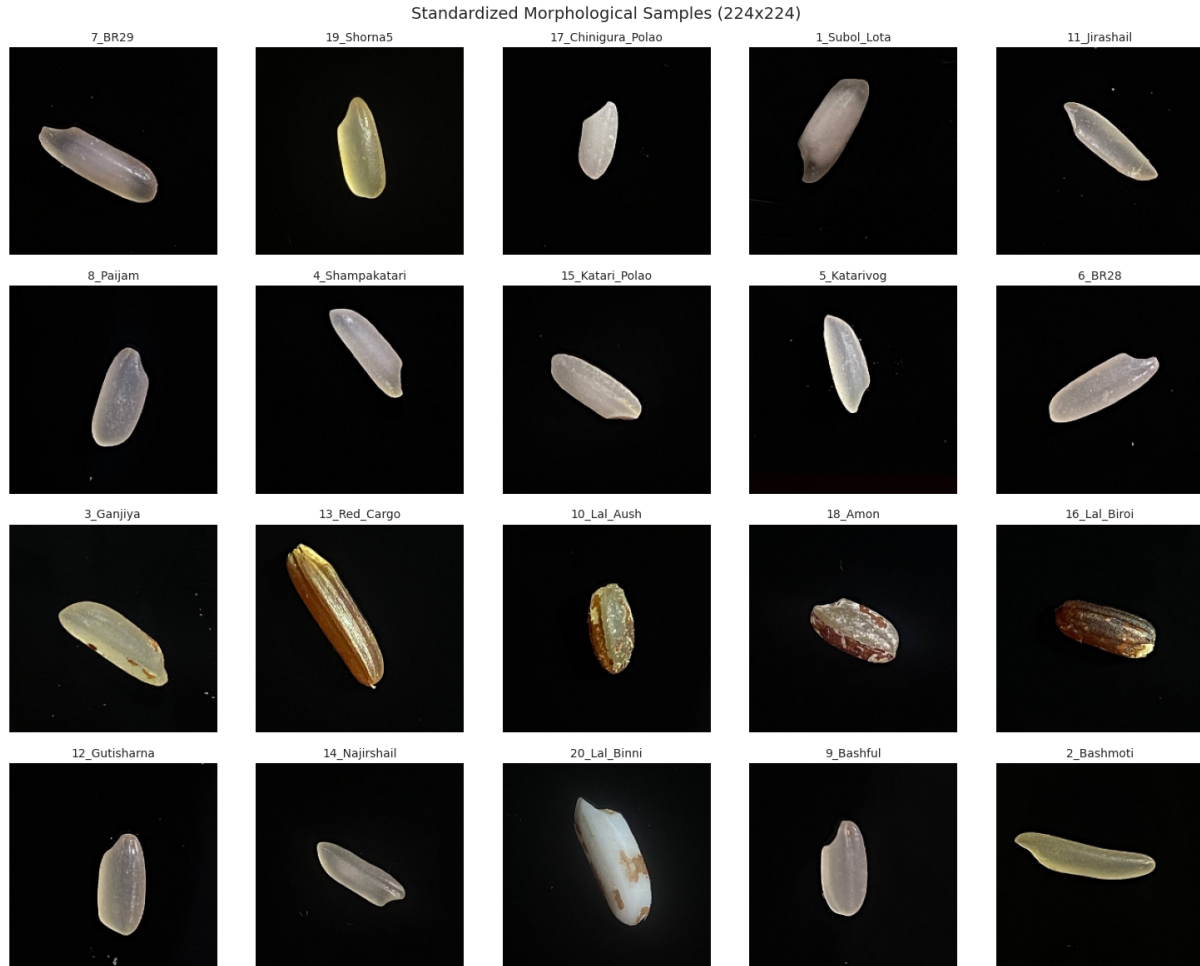


Figure 2: One representative image per rice variety, resized to 224×224 pixels. Each subtitle shows the class label.

Several visual differences are immediately apparent. Grain colour ranges from near-white (e.g., *Najirshail*) to deep red (e.g., *Lal Aush*, *Lal Binni*). Grain shape also varies noticeably, from elongated, slender grains (e.g., *Bashmoti*) to shorter, rounder ones (e.g., *Amon*). These visual cues suggest that colour and morphological features may carry discriminative information useful for classification.

2.4 Quantitative Feature Extraction

To move beyond visual inspection, a set of low-level features is computed programmatically for a stratified random sample of 2,000 images (`random_state=42`). Each image is first resized to 224×224 pixels and converted from BGR to RGB. Six scalar features are then extracted

2.4.1 RGB Channel Distributions

Figure 3 shows the kernel density estimates (KDE) of the three channel means across all sampled images.

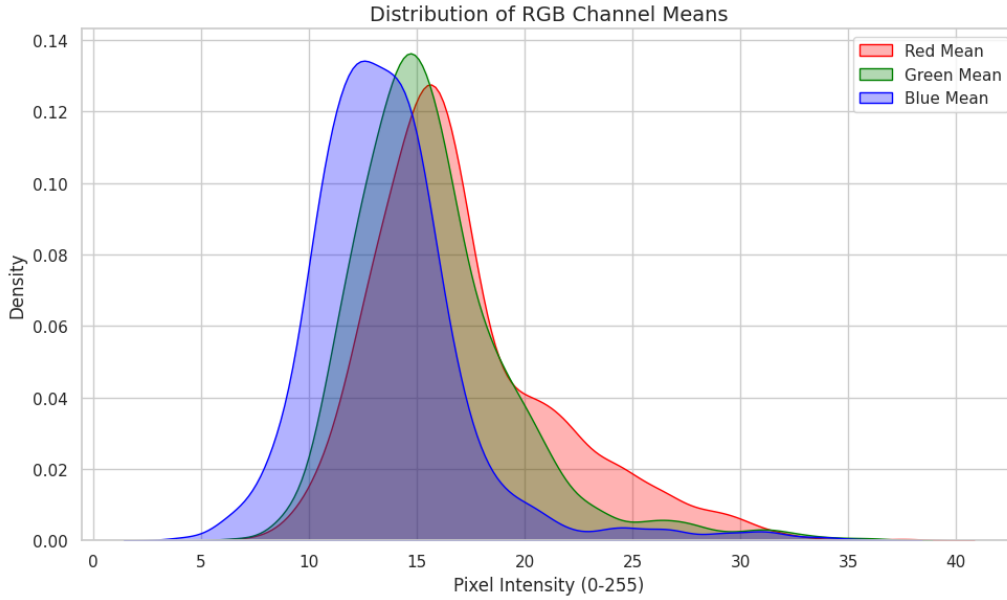


Figure 3: Kernel density estimates of the per-channel pixel intensity means (R , G , B) across 2,000 sampled images.

All three channels show broad, roughly unimodal distributions concentrated in the mid-to-high intensity range (approximately 100–200 on a 0–255 scale), which is consistent with images taken against relatively bright backgrounds. The red channel tends to have a slightly higher mean than the other two, a pattern that aligns with the warm yellowish-white colour of most rice varieties. The separation between the red and blue distributions provides a colour cue that downstream classifiers may exploit.

2.4.2 Morphological Feature Distributions by Class

Figure 4 presents box plots of grain area and aspect ratio, grouped by class.

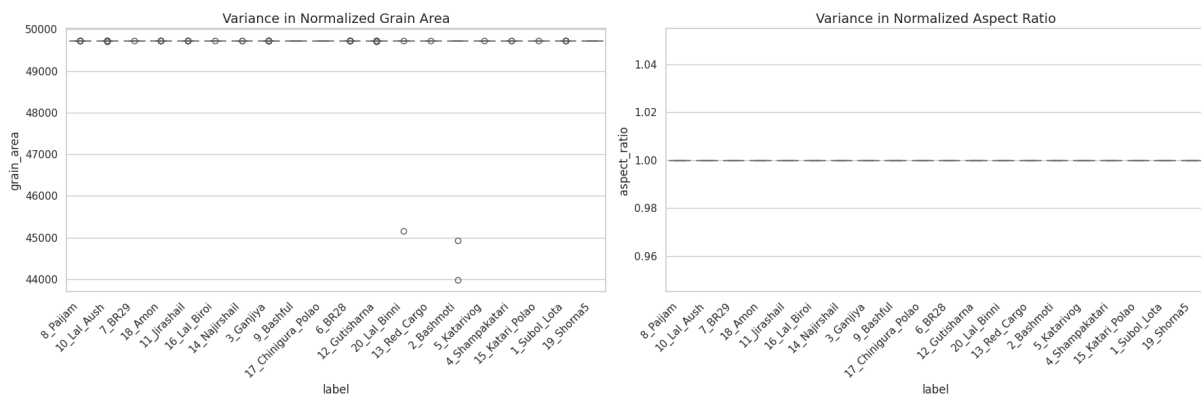


Figure 4: Box plots of (left) normalised grain area and (right) aspect ratio for each rice variety, computed on a 2,000-image sample.

Grain area varies considerably across classes. Some varieties consistently produce larger contour areas, likely because their grains are physically larger or because the camera framing

results in a closer view of the sample. Aspect ratio also differs between classes; elongated varieties such as *Bashmoti* and *Najirshail* show higher median ratios compared to rounder varieties. The inter-class spread in both features suggests they can contribute to distinguishing rice types, though within-class variance is also notable, which may reflect differences in grain orientation or the number of visible grains per image.

2.4.3 Image Quality Analysis

Figure 5 displays the five images with the lowest blur scores (blurriest) and the five with the highest (sharpest), selected from the 2,000-image sample.

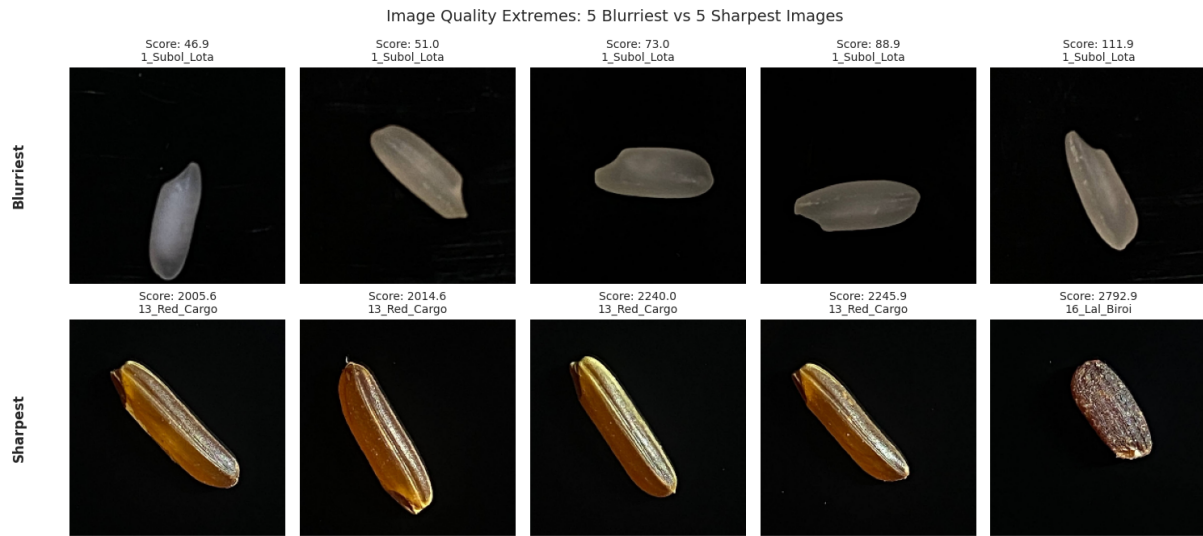


Figure 5: Image quality extremes: the five blurriest (top row) and the five sharpest (bottom row) images in the 2,000-image sample, ranked by Laplacian variance score.

The blurriest images typically show low-contrast grain boundaries and muted colours, while the sharpest images display clearly defined grain edges and fine surface texture. Because most blur scores are reasonably high across the dataset, severe focus problems are not widespread. Nevertheless, including the blur score as a feature during model training may help the classifier down-weight evidence from lower-quality images.

2.5 Feature Correlation Analysis

To examine the linear relationships among the six extracted features, a Pearson correlation matrix is computed and displayed as a heatmap in Figure 6.

Several relationships are worth noting. The three colour channel means (R, G, B) show moderate positive correlations with each other, which is expected because overall image brightness affects all channels simultaneously. Grain area and aspect ratio tend to have low correlation with colour features, indicating that they capture a different axis of variation, specifically the shape of the detected grain cluster. The blur score shows little correlation with the colour or morphological features, confirming that image sharpness is a largely independent property of image quality rather than a systematic characteristic of any particular rice variety. The relative independence between morphological and colour feature groups is favourable for classification, as it suggests the two groups together provide complementary information.

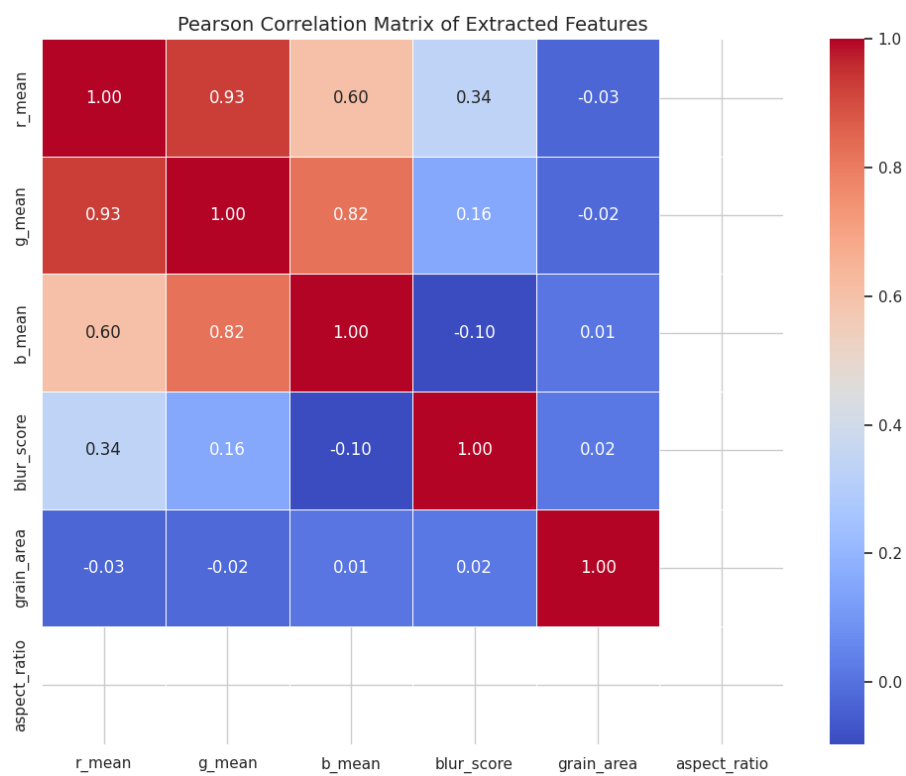


Figure 6: *Pearson correlation matrix of the six EDA features extracted from a 2,000-image sample.*

3 Preprocessing and Feature Extraction

The preprocessing pipeline plays a central role in preparing input data for advanced machine learning models. Its primary objective is to synchronize formats, optimize value ranges, and ensure absolute compatibility between real-world data and pre-trained neural network architectures. This process is carried out through four sequential steps:

3.1 Preprocess

3.1.1 Spatial Normalization

Description: Every input image, regardless of its original resolution, is resized to a single standard size (defaulting to 224×224 pixels).

Purpose: Convolutional neural network (CNN) architectures (such as ResNet or VGG) and Vision Transformers (ViT) require fixed-size matrix inputs. The 224×224 resolution is an industry standard established during the training of these models on the ImageNet dataset. Adhering to this size allows for the seamless reuse of the network structure without disrupting the spatial information processing flow.

3.1.2 Tensor Conversion

Description: The initial image format, typically a raw pixel matrix, is converted into a specialized multi-dimensional data structure known as a Tensor.

Purpose: This step reformats the data for compatibility with modern computing frameworks. The Tensor structure facilitates the maximum utilization of parallel processing capabilities on high-performance hardware such as GPUs, thereby accelerating data transfer and matrix computations during the feature extraction phase.

3.1.3 Type Casting and Scaling

Description: Original pixel values are usually integers ranging from 0 to 255. In this step, they are cast to single-precision floating-point format (32-bit float) and simultaneously scaled to fit within the range of $[0.0, 1.0]$.

Purpose: 32-bit floats are the standard requirement for calculating gradients and updating weights in gradient-based optimization. Scaling the input values to the $[0, 1]$ range ensures more stable optimization, preventing gradient explosion caused by excessively large inputs. This stabilization leads to faster and more accurate model convergence.

3.1.4 Statistical Normalization

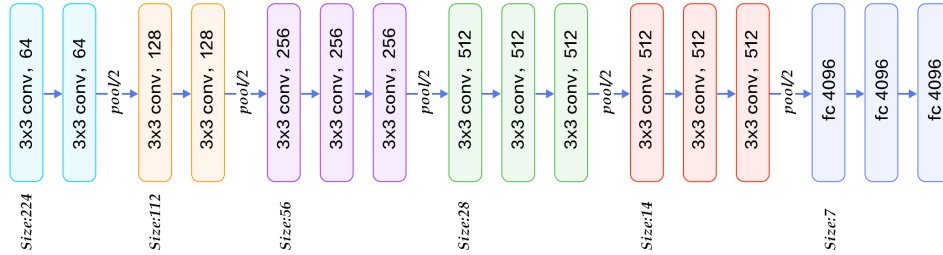
Description: The data is further normalized per color channel by subtracting a specific mean and dividing by the standard deviation. These statistical coefficients are fixed parameters derived from the ImageNet dataset: means of 0.485, 0.456, 0.406 and standard deviations of 0.229, 0.224, 0.225.

Purpose: This is a critical step when employing Transfer Learning. Since the feature extraction backbones were pre-trained on ImageNet, their filters are optimized for its specific color distribution. Applying these same statistical transformations "translates" the visual domain of the target dataset into a distribution space the model understands best. This mitigates distribution shift and ensures that the extracted features are highly accurate and meaningful.

3.2 Feature Extraction Architecture

3.2.1 VGG-16

The VGG16 network extracts visual information from rice grains based on a core trade-off mechanism: continuously reducing spatial resolution while aggressively expanding the depth (number of channels) of the features. This process begins with an input image of 224×224 pixels and 3 color channels (RGB).



3.2.1.1 Key Design Features

To optimize the extraction process without overloading the system, the architecture utilizes two fundamental techniques:

- **Small Convolutional Filters (3×3):** Stacking small filters allows the network to create a receptive field wide enough to encompass large structural details of the rice grain. More importantly, this design enables the interleaving of multiple non-linear activation functions (ReLU). In the context of rice classification, this increased non-linearity is crucial; it empowers the network to learn highly complex decision boundaries needed to distinguish between visually similar cultivars, where defining characteristics are often subtle variations in micro-texture rather than glaring differences.
- **Max Pooling Layers (2×2 window, stride 2):** Applied after convolutional blocks, this operation halves the spatial dimensions of the data matrix. While fine-grained pixel resolution is reduced, Max Pooling introduces *translation invariance*. In practical agricultural imaging, rice grains are rarely perfectly aligned or centered. This property ensures that the network successfully identifies a defining feature, such as a specific tip shape or a localized surface flaw, regardless of its absolute pixel coordinates, thereby dramatically improving the model's robustness against scattered grain distributions and varying camera setups.

3.2.1.2 Feature Extraction Process

The journey of data through consecutive convolutional blocks is divided into two main stages:

- **Initial Stage (Local Feature Detection):** Data passes through shallow layers where the number of filters increases from 64 to 128, while the spatial dimensions decrease from 224×224 to 112×112 and 56×56 . At this stage, the network acts as a sophisticated edge-detector. It focuses on isolating fundamental visual primitives: the sharp boundaries of the grain against the background, microscopic contour shifts, or minute contrast variations indicating chalkiness on the rice surface. These low-level primitives form the essential visual vocabulary for subsequent analysis.

- **Deep Stage (Semantic Feature Detection):** As the data progresses deeper, the spatial resolution is drastically compressed to 28×28 , 14×14 , and finally 7×7 , while the filter count surges to 256 and peaks at 512. This represents a critical semantic transition. The network stops "seeing" isolated geometric primitives and begins to perceive the grain as a complete entity. It synthesizes these basic building blocks to capture macro-level agronomic traits, such as the length-to-width ratio (elongation), overall morphological symmetry, and complex opacity patterns. These high, level representations are the definitive, discriminative factors required to accurately classify rice varieties that share basic physical profiles.

3.2.1.3 Strengths and Weaknesses

Strengths

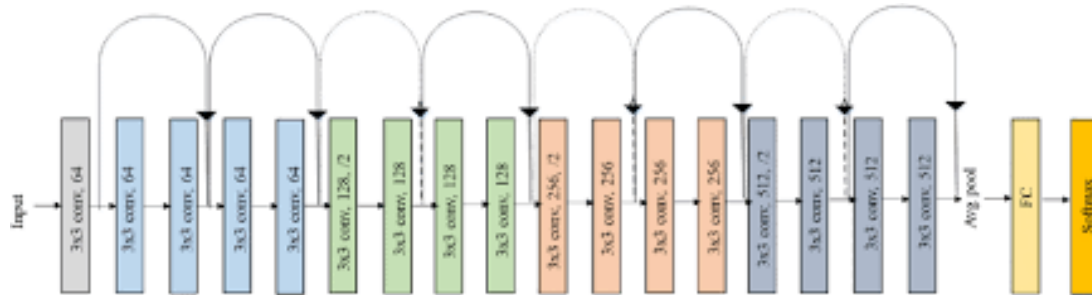
- **Rich Feature Representation:** Outputting a 4096-dimensional vector provides a highly detailed mathematical description of the input image. This depth of information is particularly effective for identifying subtle morphological differences and surface textures among various rice varieties.
- **Outstanding Transfer Learning Capability:** By being initialized with the IMAGENET1K_V1 weights, VGG16 inherits a massive "knowledge base" of basic shapes and textures. This pre-trained foundation enables the model to generalize exceptionally well on specific agricultural datasets.

Weaknesses

- **High Computational Cost and Memory Consumption:** With approximately 138 million parameters, VGG16 is an extremely bulky architecture. This leads to massive memory consumption and significantly slower inference speeds compared to more modern, optimized structures like ResNet18.
- **Curse of Dimensionality:** The massive 4096-dimensional feature space introduces challenges for downstream classification tasks. This enormous volume of data necessitates the implementation of highly robust machine learning algorithms to proactively prevent overfitting, while strictly controlling the training time during the final rice variety classification phase.

3.2.2 ResNet18

The ResNet18 (Residual Network) architecture represents a paradigm shift in deep feature extraction. ResNet18 employs a "residual learning" framework. This architecture is specifically designed to overcome the vanishing gradient problem, allowing the network to extract deeper, more complex semantic features from the rice grain images without degrading the learning process.



3.2.2.1 Key Design Features

To achieve high efficiency and accuracy, ResNet18 relies on two groundbreaking structural components:

- **Residual Blocks with Skip Connections:** As illustrated in the architecture diagram, the network features shortcut connections that bypass one or more convolutional layers. In the context of rice classification, this is revolutionary: it allows the raw, low-level structural information (such as the definitive outline of the grain) to flow directly into deeper layers. Consequently, the network can simultaneously analyze macro-level shapes and highly complex micro-textures (like slight chalkiness or surface irregularities) without the deeper layers "forgetting" the fundamental geometry of the grain.
- **Global Average Pooling (GAP):** Replacing the massive flattening operation seen in VGG16, ResNet18 utilizes an Average Pooling layer just before the final classification. This operation calculates the spatial average of each feature map. Agronomically, this enforces a strong correspondence between feature maps and specific rice traits while drastically reducing the total number of parameters (to approximately 11 million). This design makes the model highly robust to spatial translations and significantly mitigates the risk of overfitting on training data.

3.2.2.2 Feature Extraction Process

The analytical journey through ResNet18 is characterized by sequences of residual blocks, highlighted by two types of pathways:

- **Identity Mapping (Solid Arrows):** Within the same stage (e.g., 3×3 conv, 64 filters), the solid arrows indicate that the spatial dimensions and channel depths remain constant. The network refines its understanding of specific features—such as edge sharpness and localized contrast—by adding learned residuals to the preserved input features.
- **Dimensionality Reduction (Dotted Arrows):** When transitioning between stages (from 64 to 128, 256, and finally 512 channels), dotted arrows denote a downsampling process (typically via a stride of 2). Here, the spatial resolution is halved while the feature depth doubles. This forces the network to synthesize local primitives into high-level

agronomic representations, such as the overall length-to-width ratio and the holistic morphological profile of the rice cultivar.

3.2.2.3 Strengths and Weaknesses

Strengths:

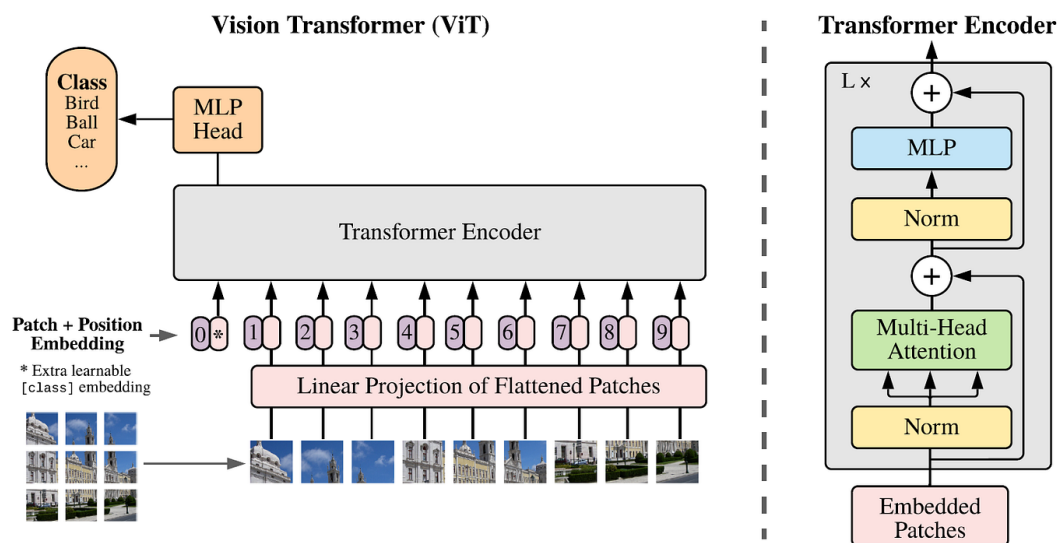
- **Optimal Balance:** Offers an exceptional balance between performance and computational efficiency.
- **Efficient Feature Extraction:** Utilizes residual connections for robust feature extraction without the immense parameter overhead.
- **Lightweight & Fast:** Its significantly smaller memory footprint makes it highly suitable for rapid inference in practical, real-time agricultural sorting systems.

Weaknesses:

- **Modification Complexity:** The rigid structure of residual blocks makes the network conceptually more complex to modify or prune at a granular level compared to straight-forward sequential models.
- **Data Dependency:** If the rice dataset is extremely small and lacks diversity, the model's deep architecture still requires careful regularization techniques to completely avoid learning dataset-specific noise.

3.2.3 Vision Transformer

The Vision Transformer (ViT) architecture fundamentally approaches visual processing by interpreting an image as a sequence of discrete patches, leveraging a purely attention-based mechanism to model global dependencies. The model processes visual data sequentially, processing sequences of image patches analogously to how standard Transformers process sequences of word embeddings.



3.2.3.1 Key Design Features

To map a 2D image into a 1D sequence format manageable by a pure attention-based architecture, ViT utilizes several foundational components, each designed with a specific functional rationale:

- **Patch Extraction and Linear Projection:** An input image $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$ is divided into a grid of non-overlapping patches. The core rationale behind this is computational feasibility: processing an image pixel-by-pixel would result in an intractable sequence length. By grouping pixels into patches, the model treats them as visual "words" (tokens), preserving local spatial structures within the patch while drastically reducing the sequence length. These patches are then flattened and mapped to a constant latent vector size D via a trainable linear projection matrix $\mathbf{E}$.
- **Learnable Class Token:** A specialized, learnable embedding vector ($\mathbf{z}_0^0 = \mathbf{x}_{class}$) is prepended to the sequence. The architectural meaning of this token is to act as an unbiased global aggregator. Because the self-attention mechanism mixes information symmetrically across all tokens, this [class] token interacts with every patch equally without being biased toward any specific spatial location. By the final layer, its state encapsulates a holistic summary of the entire image, making it the ideal representation for final classification.
- **Position Embeddings:** The pure Transformer architecture is inherently permutation-invariant; it processes sets of tokens without understanding their order. To prevent the model from treating a scrambled image identically to an ordered one, standard learnable 1D position embeddings ($\mathbf{E}_{pos}$) are added to the patch embeddings. This injects absolute spatial context back into the sequence, allowing the model to explicitly understand the topological layout of the visual features. The combined sequence is formulated as:

$$\mathbf{z}_0 = [\mathbf{x}_{class}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{pos} \quad (1)$$

3.2.3.2 Feature Extraction Process (Transformer Encoder)

The analytical core is the Transformer Encoder, consisting of L identical, stacked layers. The feature extraction is driven by two highly specialized mechanisms working in tandem:

- **Multi-Head Self-Attention (MSA):** This mechanism governs "spatial mixing." Instead of processing information locally, MSA computes attention scores between every possible pair of patches simultaneously. The significance of this is the immediate establishment of a global receptive field from the very first layer; the network dynamically learns which parts of the image are most relevant to each other, regardless of their physical distance. The "multi-head" aspect allows the model to learn multiple distinct representation subspaces (e.g., tracking geometry in one head and color consistency in another).
- **Multilayer Perceptron (MLP):** While MSA mixes information *between* different patches, the MLP governs "channel mixing." Applied to each token individually, this two-layer network with a GELU non-linearity increases the representation dimensionality and deepens the non-linear boundaries of each specific feature vector without sharing weights across the spatial sequence.
- **Structural Mechanics (Layer Normalization & Residuals):** To guarantee stable optimization in a deep architecture, Layer Normalization (LN) is applied immediately *before*

every MSA and MLP block. This "pre-norm" design ensures that feature scales remain consistent. Furthermore, residual connections bypass these blocks, acting as information superhighways that prevent the degradation of earlier learned features and maintain smooth gradient flow. The extraction dynamics for layer l are mathematically defined as:

$$\mathbf{z}'_l = \text{MSA}(\text{LN}(\mathbf{z}_{l-1})) + \mathbf{z}_{l-1} \quad (2)$$

$$\mathbf{z}_l = \text{MLP}(\text{LN}(\mathbf{z}'_l)) + \mathbf{z}'_l \quad (3)$$

3.2.3.3 Strengths and Weaknesses

Strengths:

- **Global Context Modeling:** The self-attention mechanism enables the network to integrate global information from the very first layer, capturing long-range dependencies efficiently.
- **Massive Scalability:** The architecture exhibits exceptional scalability, showing continuous performance improvements as model size and pre-training data volume increase without rapidly saturating.

Weaknesses:

- **Data Hunger:** The model lacks inherent inductive biases (such as translation invariance or local connectivity). Consequently, it requires massive amounts of training data to learn these properties from scratch effectively.
- **Quadratic Complexity:** The computational cost of the self-attention mechanism scales quadratically with the sequence length (number of patches). Processing high-resolution images generates a massive sequence, leading to extreme memory consumption and high computational overhead.

4 Machine Learning Training

4.1 Algorithmic Selection and Implementation Strategies

To comprehensively evaluate the discriminative power of the deep learning embeddings extracted in the previous stage, the pipeline implements a robust suite of five traditional machine learning algorithms. Let $z \in \mathbb{R}^d$ denote the dense feature vector extracted from an input image by a frozen pretrained backbone. The objective of this secondary training phase is to learn a mapping from the continuous embedding space z to a discrete label space representing the 20 rice varieties.

The selected algorithms span parametric, non-parametric, linear, and ensemble methods, ensuring a thorough evaluation of the feature space geometry:

- **Logistic Regression:** Serving as the baseline linear model, logistic regression fits a multinomial decision boundary. Because linear algorithms are highly sensitive to the magnitude and variance of input features, the raw embeddings z are strictly normalized using a standard scaler, transforming them to have zero mean and unit variance ($z' = \frac{z - \mu}{\sigma}$). In environments utilizing the custom deep-learning backend, this is implemented as a single-layer neural network optimized via the Adam optimizer with a learning rate of 0.05, a weight decay of 10^{-4} , and the cross-entropy loss objective over 250 epochs.
- **Linear Support Vector Machine (Linear SVM):** A max-margin classifier explicitly designed to find the optimal separating hyperplane between classes. A linear kernel is utilized exclusively; given that the feature embeddings are already highly dimensional ($d \geq 512$), projecting them into an even higher-dimensional space using non-linear kernels (e.g., RBF) is mathematically redundant and computationally prohibitive. In the PyTorch-accelerated implementation, the SVM formulation is approximated using Stochastic Gradient Descent (SGD) with a Nesterov momentum of 0.9. The model optimizes a `MultiMarginLoss` objective over 300 epochs, where the traditional regularization parameter C is inversely mapped to the optimizer's weight decay.
- **k -Nearest Neighbours (k -NN):** A non-parametric distance-based classifier that assigns labels based on the majority vote of the k closest samples in the training set. Standard scaling is applied to guarantee that features with larger numerical ranges do not disproportionately dominate the Euclidean distance calculations. A significant engineering challenge with k -NN in high-dimensional feature spaces is the immense memory footprint of pairwise distance matrices ($\mathcal{O}(N^2)$). To mitigate out-of-memory errors on accelerators, the custom implementation computes distances in constrained chunks (batch size of 512) iteratively.
- **Random Forest:** A powerful ensemble learning technique that constructs a multitude of unpruned decision trees during training. Predictions are aggregated via a majority vote. Random Forests inherently partition the feature space using orthogonal splits, making them invariant to monotonic transformations; thus, standard scaling is bypassed for this model. It provides a strong non-linear baseline that is highly resistant to overfitting on high-dimensional data.
- **XGBoost:** An advanced implementation of gradient boosted decision trees. The pipeline utilizes XGBoost to sequentially minimize a multi-class logarithmic loss objective (`multi:softmax` driven by `mlogloss`). To accelerate training on the large arrays of extracted features, the model is configured to use histogram-based tree methods (`tree_method="hist"`), which discretize continuous features into bins, vastly reducing the computational complexity of evaluating split candidates.

4.2 Hardware Acceleration and Backend Routing

A defining characteristic of the implemented pipeline is its dynamic hardware-acceleration routing, designed to process large embedding arrays efficiently. Training classical models on thousands of 512-dimensional vectors is computationally intensive on standard CPUs. To address this, the pipeline includes an automatic device-resolution module (`init_backend`) that seamlessly pivots between three distinct computational backends:

1. **cuML / CuPy (CUDA):** When NVIDIA hardware is detected, the pipeline preferentially routes execution to the RAPIDS `cuml` library. This allows models like Random Forest, SVM, and k -NN to be trained entirely on the GPU, achieving massive parallelization.
2. **Custom PyTorch (CUDA/MPS):** If `cuml` is unavailable but GPU hardware (including Apple Silicon MPS) is present, the system falls back to custom PyTorch implementations of Logistic Regression, SVM, and k -NN (`TorchLogRegClassifier`, `TorchLinearSVMClassifier`, etc.). These proxy models utilize gradient descent and tensor operations to mimic classical algorithms while benefiting from hardware acceleration.
3. **Scikit-Learn (CPU):** In the absence of compatible accelerators, the pipeline defaults to standard `scikit-learn` implementations executed on the CPU.

This dynamic backend abstraction ensures that the models scale efficiently regardless of the underlying hardware constraints, drastically reducing the time required for comprehensive hyperparameter sweeps.

4.3 Hyperparameter Tuning Protocol

To ensure a rigorous and fair comparison across different algorithms and feature backbones, the system employs an automated grid-search hyperparameter tuning strategy. Models are iteratively fitted on the training split ($\mathcal{D}_{\text{train}}$) and evaluated on a mutually exclusive validation split ($\mathcal{D}_{\text{val}}$).

The hyperparameter search space is predefined to test the bounds of model capacity and regularization:

- **Linear Models (Logistic Regression & SVM):** The inverse regularization strength C is swept across $\{0.1, 1.0, 3.0\}$ to control the trade-off between maximizing the classification margin and penalizing training errors.
- **k -NN:** Evaluated across varying neighbourhood sizes $k \in \{3, 5, 9\}$ and voting algorithms (uniform vs. distance-weighted).
- **Random Forest:** Tuned across ensemble sizes $N \in \{200, 300, 400\}$ and maximum tree depths $D \in \{20, 30, 40\}$, with the maximum features considered per split constrained to $\sqrt{d}$.
- **XGBoost:** The learning rate $\eta \in \{0.05, 0.1\}$ is cross-validated against maximum tree depths $D \in \{6, 8\}$ and estimator counts $N \in \{200, 300, 400\}$. Furthermore, stochastic regularization is introduced by tuning row (`subsample`) and column (`colsample_bytree`) sampling rates between 0.9 and 1.0.

Selection Criteria: The definitive configuration for a given model is selected based on the highest validation accuracy achieved during the grid search. Because the grid can produce identical accuracy scores across slightly different configurations, the pipeline explicitly utilizes

the validation **macro F1-score** as a secondary tie-breaking metric. This ensures that the chosen configuration not only maximizes overall correctness but also maintains the most balanced predictive performance across all individual classes. Additionally, the time taken to evaluate the grid (**tuning_time**) is logged to benchmark the computational efficiency of the respective backends.

4.4 Evaluation Metrics

Following hyperparameter optimization, the globally optimal models are isolated and evaluated on a strictly unseen test set ($\mathcal{D}_{\text{test}}$). This final evaluation provides an unbiased estimate of the model's generalization capabilities on novel rice grain imagery.

Let y denote the ground truth labels and $\hat{y}$ denote the model predictions. The evaluation module calculates a rigorous suite of metrics using **scikit-learn**:

- **Accuracy:** The standard measure of correctness, defined as the ratio of correctly predicted instances to the total number of instances.
- **Macro Precision:** The arithmetic mean of the precision scores calculated independently for each of the 20 classes. Precision for a single class c is defined as $\frac{TP_c}{TP_c + FP_c}$.
- **Macro Recall:** The arithmetic mean of the recall scores calculated independently for each class. Recall for class c is $\frac{TP_c}{TP_c + FN_c}$.
- **Macro F1-score:** Computed as the unweighted harmonic mean of the per-class precision and recall.

The explicit use of macro-averaging (**average="macro"**) in Precision, Recall, and F1-score is a critical methodological choice. In agricultural datasets, sample distributions across varieties are occasionally imbalanced. Macro-averaging treats all 20 rice varieties equally, regardless of their support size in the test set. Consequently, a model cannot achieve a high macro F1-score simply by over-optimizing for the most common rice varieties; it is mathematically forced to perform well on the difficult, less-frequent, and visually ambiguous classes.

4.5 Results and Comparison

4.5.1 Overall Performance by Feature Representation

The evaluation of traditional machine learning models on deep feature embeddings confirms that the choice of pretrained backbone dictates the upper bound of downstream classification performance. As summarized in Table 1, across all five traditional algorithms evaluated, the embeddings extracted by the ViT-B/16 architecture consistently yielded the highest predictive accuracy and class separability.

Traditional models trained on ViT features achieved a peak test accuracy of 91.86% (using Logistic Regression), with the macro F1-score (91.85%) remaining tightly coupled to accuracy. This minimal gap between accuracy and macro F1 indicates that the transformer-based self-attention mechanism produces a highly uniform embedding space that effectively resolves minority and visually ambiguous rice varieties.

Conversely, models trained on ResNet18 features achieved strong but comparatively lower peak performance, reaching 87.84% test accuracy with an 87.88% macro F1-score. VGG16 embeddings provided the weakest representational foundation, peaking at 85.31% test accuracy. These trends strictly parallel the representational hierarchy observed during the deep neural head training phase, confirming that classical models are highly sensitive to the geometric quality of the input feature space.

Feature Backbone	Best Model	Val Acc.	Val F1	Test Acc.	Test F1	Fit Time (s)
ViT-B/16	Logistic Regression	91.75%	91.76%	91.86%	91.85%	0.89
ResNet18	Logistic Regression	90.27%	90.34%	87.84%	87.88%	3.76
VGG16	Logistic Regression	86.15%	86.03%	85.31%	85.28%	0.91

Table 1: *Best performing traditional model for each pretrained feature architecture.*

4.5.2 Evaluation of Traditional Classifiers

A comprehensive analysis of the benchmarking data (presented in Table 2) reveals distinct behavioural differences among the traditional classifiers when operating on high-dimensional dense vectors:

- **Logistic Regression & Linear SVM:** These linear models emerged as the strongest and most robust traditional baselines. Notably, Logistic Regression, operating on standard-scaled features, was the absolute best-performing traditional model across all three architecture backbones. The success of linear boundaries strongly implies that the pretrained deep learning backbones successfully disentangle the rice varieties into linearly separable clusters. The Linear SVM followed closely behind (e.g., 82.98% test accuracy on ViT features) but trailed the logistic cross-entropy objective.
- **k -Nearest Neighbours (k -NN):** While k -NN established a reasonable non-parametric baseline, it generally trailed behind the max-margin classifiers, achieving 80.55% test accuracy on the best-case ViT features. In the high-dimensional embedding space ($d \geq 512$), distance metrics become less discriminative due to the curse of dimensionality.
- **Tree Ensembles (Random Forest & XGBoost):** Despite their sophisticated capacity for nonlinear modelling, Random Forest and XGBoost did not surpass the linear baselines. Random Forest proved resistant to overfitting but struggled to effectively partition the continuous space using orthogonal splits, maxing out at 84.46% on ResNet18 and 84.25% on ViT features. XGBoost achieved a lower test accuracy of 87.32% on ViT features compared to the 91.86% achieved by Logistic Regression. This reinforces the conclusion that the classification problem at this stage does not require complex non-linear decision trees.

4.5.3 Computational Efficiency and Hardware Acceleration

Beyond predictive accuracy, the benchmarking pipeline highlighted massive disparities in computational efficiency. The implementation of dynamic hardware routing (utilizing `cuda_torch`) proved critical for scaling the grid-search hyperparameter tuning.

Linear models demonstrated exceptional computational economy. Logistic regression converged rapidly, requiring only 0.89 seconds of tuning time for the ViT embeddings, and 3.76 seconds for ResNet18. In stark contrast, training tree-based ensembles on thousands of dense, high-dimensional vectors was computationally bottlenecked. Even with CUDA acceleration, XGBoost required substantially longer fit times—taking 97.73 seconds for ViT features and a massive 272.12 seconds for VGG16 features—while failing to yield a proportional increase in predictive power.

Ultimately, Logistic Regression offers the optimal trade-off in the traditional machine learning suite: it achieves near state-of-the-art accuracy on transformer embeddings, requires less than a second of computational overhead for tuning, and provides highly interpretable decision boundaries.

Backbone	Model	Val Acc.	Val F1	Test Acc.	Test F1	Fit Time (s)
ViT-B/16	Logistic Regression	91.75%	91.76%	91.86%	91.85%	0.89
	XGBoost	86.15%	85.96%	87.32%	87.27%	97.73
	Random Forest	84.46%	84.09%	84.25%	84.10%	12.68
	Linear SVM	84.78%	84.79%	82.98%	83.23%	0.76
	k -NN	80.55%	80.34%	80.55%	80.63%	0.02
ResNet18	Logistic Regression	90.27%	90.34%	87.84%	87.88%	3.76
	XGBoost	87.10%	87.08%	85.20%	85.19%	77.34
	Random Forest	86.26%	85.89%	84.46%	84.25%	10.34
	Linear SVM	80.97%	81.13%	80.13%	80.49%	0.78
	k -NN	79.28%	79.27%	79.28%	79.40%	0.10
VGG16	Logistic Regression	86.15%	86.03%	85.31%	85.28%	0.91
	XGBoost	84.46%	84.38%	83.51%	83.57%	272.12
	Linear SVM	82.24%	82.28%	82.77%	83.13%	0.90
	Random Forest	81.08%	80.63%	79.81%	79.67%	9.84
	k -NN	75.58%	75.17%	73.47%	73.36%	0.07

Table 2: Performance comparison of all traditional classifiers across different feature backbones.

5 Deep Learning Training

5.1 Transfer Learning Strategy

In this work, transfer learning is used to classify images from the Mendeley rice-variety dataset. Rather than training a deep convolutional neural network entirely from scratch, the model pipeline reuses visual representations learned from large-scale natural-image data. This choice is motivated by two practical considerations. First, the dataset contains a limited number of original labelled images compared with the amount of data typically required to train a high-capacity deep model end-to-end. Second, the task involves fine-grained visual discrimination across 20 rice varieties, where subtle differences in colour, shape, texture, and grain arrangement must be captured reliably. Transfer learning provides a more stable and data-efficient solution under these conditions [2].

Let $x \in \mathbb{R}^{H \times W \times 3}$ denote an RGB input image resized to the target resolution of 224×224 pixels. A pretrained backbone network acts as a feature extractor and maps the image into a compact latent representation,

$$z = \phi(x; \theta_{\text{base}}),$$

where $\phi(\cdot)$ denotes the convolutional backbone and θ_{base} denotes the pretrained backbone parameters. In the implemented pipeline, several alternative backbones are supported through the shared model-building interface, and feature extraction is performed for each backbone separately. For every split in $\{\text{train}, \text{val}, \text{test}\}$, the extracted feature vectors are saved to disk as NumPy arrays before classifier training. This means that the backbone is used in inference mode to produce fixed embeddings, while the downstream classifier is trained independently on those embeddings.

The classifier itself is a lightweight fully connected head,

$$\hat{y} = g(z; \theta_{\text{head}}),$$

where $\hat{y} \in \mathbb{R}^C$ is the predicted class score vector and C is the number of rice varieties. For this

dataset, $C = 20$. As shown in the implementation, the classification head consists of two linear layers with an intermediate hidden representation of size 512, followed by batch normalization, a ReLU activation, and dropout regularization. The final layer produces logits for the 20 classes, and these logits are optimized using the cross-entropy loss

$$\mathcal{L}_{\text{CE}} = - \sum_{c=1}^C y_c \log(\hat{p}_c),$$

where y_c is the ground-truth indicator for class c and $\hat{p}_c$ is the predicted probability for that class after the softmax operation.

An important detail is that the backbone parameters are not updated during classifier training. Instead, the learning problem is reduced to optimizing only the classifier-head parameters,

$$\theta_{\text{head}} \leftarrow \theta_{\text{head}} - \eta \nabla_{\theta_{\text{head}}} \mathcal{L},$$

while θ_{base} remains fixed after offline feature extraction. This design is fully consistent with the provided code: pretrained backbones first generate train, validation, and test features, and a separate `SoftmaxClassifierHead` is then trained on these stored features using the Adam optimizer. The approach substantially reduces computational cost, allows rapid comparison across multiple backbone architectures, and limits the risk of overfitting because only a small number of trainable parameters are learned in the final stage.

To improve optimization stability, the training loop monitors both training and validation performance over epochs. Early stopping is applied based on the validation loss with a patience window, while the best-performing checkpoint is selected according to validation accuracy. After training, the saved head is evaluated on the test split using accuracy, weighted precision, weighted recall, weighted F1-score, and the confusion matrix. This evaluation protocol is appropriate for the present task because it not only measures overall correctness but also reveals how effectively each pretrained backbone separates visually similar rice varieties.

Overall, the transfer learning strategy using is best described as *feature-based transfer learning*. Knowledge learned from large generic image collections is transferred through pretrained convolutional backbones, and these representations are then adapted to the rice-variety classification task by training a compact neural classifier on top of extracted features. This makes the deep learning stage efficient, reproducible, and well aligned with the structure of the implemented code.

5.2 Backbone Architectures

Figure 7 provides a compact visual summary of the three pretrained backbones used in this study. The diagram highlights the main structural differences among the models: ResNet18 relies on residual convolutional stages [3], VGG16 uses a deeper sequential stack of convolutional blocks [4], and ViT-B/16 processes the image as a sequence of fixed-size patches through self-attention [5]. This visualization is useful because it makes clear that all three backbones serve the same role in the pipeline—feature extraction—while arriving at that representation through very different architectural principles.

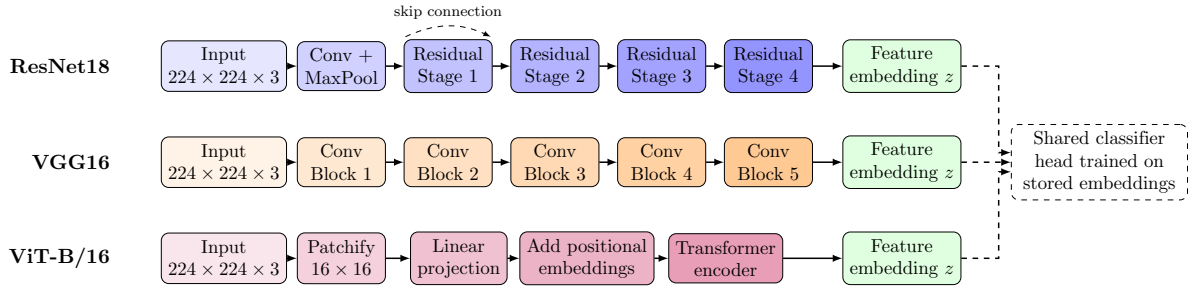


Figure 7: Compact visualization of the three pretrained backbone architectures used for feature extraction. ResNet18 employs residual stages with skip connections, VGG16 uses a strictly sequential stack of convolutional blocks, and ViT-B/16 converts the image into patch tokens processed by a transformer encoder before producing the embedding used by the shared classifier head.

5.2.1 ResNet18

ResNet18 is a convolutional neural network built on the principle of residual learning and is one of the most widely used compact members of the ResNet family [3]. Instead of forcing each stack of layers to learn a complete transformation directly, residual blocks learn only the difference between the input and the desired output. If x denotes the block input and $H(x)$ the underlying mapping to be learned, the residual formulation writes

$$F(x) = H(x) - x,$$

so that the final block output becomes

$$y = F(x) + x.$$

As illustrated in Figure 7, the ResNet18 branch begins with the $224 \times 224 \times 3$ input image, passes it through an initial convolution and max-pooling stage, and then successively processes the feature maps through four residual stages before producing the final embedding z . The dashed skip-connection drawn above the residual stages emphasizes the defining architectural idea of ResNet: part of the input representation is forwarded directly to later layers and combined with the learned residual mapping. These shortcut connections improve gradient flow during backpropagation, reduce optimization difficulty, and make it possible to train deeper networks more reliably than plain convolutional architectures. In the present rice-variety classification task, this property is useful because the model must preserve subtle visual cues across many layers while still remaining computationally efficient.

Structurally, ResNet18 contains an initial convolutional layer and max-pooling stage, followed by four main residual stages that progressively increase channel depth while reducing spatial resolution. Unlike deeper bottleneck-based variants such as ResNet50, ResNet18 uses the simpler basic residual block with two 3×3 convolutions per block. Early layers capture low-level patterns such as grain edges, colour contrasts, and local texture, whereas deeper layers encode more abstract shape configurations, surface patterns, and arrangements that help distinguish visually similar rice varieties. This hierarchical feature extraction is well suited to fine-grained recognition problems, where small appearance differences must be separated consistently.

Within the implemented pipeline, ResNet18 is not fine-tuned end-to-end. Instead, the pre-trained ImageNet backbone is used as a fixed feature extractor, and its output embeddings are stored before classifier-head training. This makes ResNet18 an attractive backbone because it offers a strong trade-off between representational quality, computational cost, and stability. As

shown later in the experimental results, the ResNet18 features produce highly robust downstream performance with only a very small validation–test gap, indicating that the learned representation generalizes well to unseen rice-grain images.

5.2.2 Vgg16

VGG16 is a classical deep convolutional architecture known for its simple and uniform design. The network consists of 16 weight layers arranged as sequential stacks of small convolutional filters, primarily 3×3 , interleaved with max-pooling operations for spatial downsampling. Rather than using complex branching or shortcut connections, VGG16 increases representational power by stacking many convolutional layers in depth. This design demonstrated that repeated use of small filters can capture rich visual structure while keeping the architectural pattern straightforward and easy to interpret.

The architecture diagram in Figure 7 shows this design as a strictly linear progression from the input image through five convolutional blocks and finally to the extracted embedding z . Unlike ResNet18, no shortcut arrows or parallel paths appear in the VGG16 row, which visually reinforces that information flows through a purely sequential stack of learned filters. The architecture is typically organized into five convolutional blocks, each containing multiple convolutional layers followed by pooling, and then a classifier module in the original ImageNet formulation. In transfer-learning applications such as the present study, the classification part is replaced or discarded, and the convolutional backbone serves only as a feature encoder. The earlier VGG16 layers tend to learn primitive image cues such as edges, contours, and basic texture patterns, while the deeper layers respond to more complex visual arrangements. For rice-grain images, these progressively learned features can represent differences in elongated shape, local texture density, colour tone, and grain-surface appearance that are relevant to variety recognition.

Although VGG16 is older and less parameter-efficient than more modern backbones, it remains an important reference model because its feature maps are often strong and stable in transfer-learning settings. In this work, VGG16 is used in exactly the same feature-based transfer-learning framework as the other backbones: pretrained weights provide fixed embeddings, and the custom neural head is trained separately on the extracted vectors. This allows VGG16 to function as a meaningful baseline for comparison against both residual networks and transformer-based models. The later results show that VGG16 remains competitive, although its final performance is lower than that of ResNet18 and ViT-B/16, suggesting that the main limitation lies in representation quality rather than in the downstream classifier.

5.2.3 Vit_b_16

ViT-B/16 (Vision Transformer Base with 16×16 patch size) represents a different family of visual backbone compared with conventional convolutional networks. Instead of processing the image through local convolutional kernels, the model first divides the input image into a sequence of non-overlapping patches, each of size 16×16 . After flattening and linear projection, these patches are treated as token embeddings and passed through a transformer encoder. Positional embeddings are added so that spatial order is preserved, and self-attention layers then model relationships among all patches across the image. This mechanism enables the network to capture both local and long-range dependencies in a unified representation.

This processing sequence is also visible in Figure 7, where the ViT-B/16 row moves from the

input image to patchification, linear projection, addition of positional embeddings, and then a transformer encoder before yielding the feature embedding z . The figure is helpful because it makes clear that the ViT backbone does not aggregate information through progressively deeper convolutional stages; instead, it converts the image into tokens and models their relationships through attention. Formally, if an image $x \in \mathbb{R}^{224 \times 224 \times 3}$ is partitioned into N patches, the transformer processes the token sequence by repeatedly applying multi-head self-attention and feed-forward layers. Unlike CNNs, where receptive fields expand gradually with depth, ViT can directly relate distant image regions even in early layers. For fine-grained rice-variety classification, this is particularly valuable because discrimination may depend not only on local texture or colour cues, but also on broader shape proportions and the spatial arrangement of grain structures. The transformer representation therefore has the potential to encode subtle global-and-local interactions more effectively than purely convolutional alternatives.

In the implemented pipeline, ViT-B/16 is again used as a frozen pretrained feature extractor rather than as a fully fine-tuned end-to-end classifier. The extracted embeddings are saved and subsequently passed to the same lightweight classification head used for the CNN backbones, ensuring a fair comparison across architectures. Among the evaluated backbones, ViT-B/16 yields the strongest overall performance in this study. Its superior validation and test metrics suggest that the patch-based self-attention representation captures the fine-grained visual differences between rice varieties especially well, making it the most effective backbone in the present transfer-learning framework.

5.3 Custom Classification Head

Unlike end-to-end designs that attach pooling modules directly to the last convolutional feature map, the implemented pipeline trains the classifier on pre-extracted backbone embeddings saved as NumPy arrays. For each pretrained backbone, the feature-extraction stage first produces a fixed vector $z \in \mathbb{R}^d$ for every rice-grain image in the train, validation, and test splits. Here, d depends on the selected backbone architecture after its output has been flattened into a one-dimensional representation. The role of the custom classification head is therefore to map this backbone-specific embedding into logits over the 20 rice-variety classes of the Mendeley Aruzz22.5K dataset, rather than to perform additional spatial pooling on raw feature maps.

Formally, let

$$z = \phi(x; \theta_{\text{base}}) \in \mathbb{R}^d$$

denote the fixed feature vector extracted from an input rice image $x \in \mathbb{R}^{224 \times 224 \times 3}$ by a pretrained backbone $\phi(\cdot)$. The classifier head then computes

$$h = \sigma(\text{BN}(W_1 z + b_1)),$$

followed by dropout regularization,

$$\tilde{h} = \text{Dropout}(h; p),$$

and finally the class logits,

$$\hat{y} = W_2 \tilde{h} + b_2,$$

where $W_1 \in \mathbb{R}^{512 \times d}$, $W_2 \in \mathbb{R}^{20 \times 512}$, $\text{BN}(\cdot)$ denotes batch normalization, $\sigma(\cdot)$ is the ReLU activation, and $p = 0.2$ is the dropout rate used in the code. This structure corresponds exactly

to the `SoftmaxClassifierHead` module, which consists of two fully connected layers with an intermediate hidden dimension of 512.

The design is intentionally lightweight. Because the pretrained backbone has already encoded much of the visual information needed to distinguish rice varieties, the downstream head only needs enough capacity to separate the resulting feature vectors in the target label space. Batch normalization helps stabilize optimization by reducing internal covariate shift across mini-batch embeddings, while the ReLU nonlinearity allows the classifier to learn more expressive class boundaries than a purely linear mapping. Dropout is included to reduce overfitting, which remains important because the final classifier is trained on a task with fine-grained inter-class similarity among visually related rice grains.

This implementation should be interpreted as a compact multilayer perceptron head rather than a global-pooling head. Operations such as Global Average Pooling (GAP), Global Max Pooling (GMP), or multi-scale fusion are not learned inside the classifier stage described here. Instead, any spatial aggregation is implicitly handled by the pretrained backbone and the feature-extraction step. Once the stored features are loaded through `TensorDataset` and `DataLoader`, the head receives only tabular feature vectors of shape $N \times d$, where N is the number of samples in the corresponding split.

The final output layer produces 20 logits, one for each rice variety in the dataset. During training, these logits are passed directly to the cross-entropy objective implemented by `nn.CrossEntropyLoss`, which internally combines the softmax transformation with negative log-likelihood optimization. Prediction is then obtained by selecting the class with maximum logit value,

$$\hat{c} = \arg \max_{c \in \{1, \dots, 20\}} \hat{y}_c.$$

This makes the head fully compatible with the evaluation pipeline, where test predictions are used to compute accuracy, weighted precision, weighted recall, weighted F1-score, and the confusion matrix for each backbone.

Overall, the custom classification head is simple but well suited to the architecture used in this study. It preserves the efficiency advantages of offline feature extraction, supports fair comparison across multiple pretrained backbones, and provides enough discriminative capacity to adapt generic visual representations to the 20-class rice-variety recognition problem.

5.4 Training and Validation Performance

Training and validation behaviour was analysed for the three pretrained backbones (`resnet18`, `vgg16`, and `vit_b_16`). Across all experiments, the classifier head learned rapidly during the first few epochs, with both loss reduction and accuracy improvement occurring early in training. This pattern is significant because it indicates that the frozen backbone features already organize the rice images into a relatively separable latent space. In other words, the classifier head is not learning low-level visual features from scratch; instead, it primarily learns decision boundaries between classes that are already partially disentangled by the pretrained representation. This behaviour is consistent with the expected outcome of a successful feature-based transfer-learning pipeline and further supports the computationally efficient

design choice of training only the classification head rather than fine-tuning the entire backbone.

Among the evaluated models, the Vision Transformer backbone produced the best overall learning dynamics. Its validation accuracy increased from 87.53% in the first epoch to a peak of 93.87%, while the validation loss fell from approximately 0.403 to a minimum near 0.181 before stabilizing. The corresponding test results confirmed that this strong validation behaviour translated well to unseen data, yielding a test accuracy of 92.18%, weighted precision of 92.52%, weighted recall of 92.18%, and weighted F1-score of 92.07%. Beyond the numerical advantage, the more meaningful point is that ViT features appear to preserve fine-grained structure between visually similar rice varieties better than the CNN-based alternatives. Rice-grain recognition depends on subtle cues such as texture distribution, elongated shape differences, and local colour variation; the stronger ViT result suggests that its patch-based self-attention representation captures these global-and-local relations in a way that makes the final classes easier to separate. The relatively small validation–test drop of about 1.69 percentage points further indicates that this superior validation behaviour is not merely a training artefact, but reflects genuinely transferable discriminative information in the learned feature space.

The ResNet18-based pipeline showed the most stable training behaviour. Training accuracy increased steadily from 74.77% to 98.94%, while validation accuracy improved quickly and then remained close to the 90% level for most of the run. Its best validation accuracy reached 90.59%, and the final test accuracy was 90.49%, with weighted precision, recall, and F1-score of 91.09%, 90.49%, and 90.46%, respectively. Notably, the difference between best validation accuracy and test accuracy was only about 0.11 percentage points, which suggests that the ResNet18 features produced a very robust classifier with minimal overfitting. This stability is analytically important: although ResNet18 does not achieve the highest peak accuracy, it offers the most predictable validation-to-test transfer, meaning that its embeddings are not only discriminative but also consistent across splits. For an applied classification system, such consistency can be nearly as valuable as raw peak accuracy, because it implies lower sensitivity to sampling variation and a smaller risk that the model is exploiting split-specific feature patterns. In that sense, ResNet18 behaves like the most reliable backbone, even if ViT-B/16 is the strongest one overall.

The VGG16 backbone achieved competitive but weaker performance than the other two extractors. Validation accuracy improved from 82.35% in the first epoch to a peak of 89.64%, and the validation loss decreased overall, although its curve fluctuated more strongly in later epochs. On the test set, VGG16 reached an accuracy of 86.05% with a weighted F1-score of 86.05%. Compared with ResNet18 and ViT-B/16, this model exhibited the largest validation–test gap (3.59 percentage points), indicating that its extracted features were less stable for final generalization in this rice-variety classification task. The interpretation here is not simply that VGG16 is “worse” in a numerical sense, but that its representation seems less well aligned with the subtle structure of this dataset. The stronger late-epoch fluctuation suggests that the head can fit the training and validation signals to some extent, yet the resulting boundaries are less robust when transferred to the held-out test set. This is consistent with the broader view that earlier CNN families such as VGG16 provide useful generic visual descriptors, but may be less efficient at encoding the kind of fine-grained inter-class variation that modern residual and transformer architectures capture more effectively.

Taken together, the training histories show that all three classifier heads are trainable and effective when built on top of frozen pretrained embeddings, but their final discriminative

quality depends strongly on the backbone. In terms of both convergence quality and test performance, the ranking is ViT-B/16 > ResNet18 > VGG16. More importantly, the differences between them reveal three distinct representation behaviours: ViT-B/16 provides the richest and most separable feature space, ResNet18 provides the most stable and reliable generalization, and VGG16 provides a usable but comparatively weaker embedding space. Therefore, the learning curves do more than identify the best score; they show how backbone choice changes the balance between peak accuracy, optimization smoothness, and generalization reliability in fine-grained rice-variety recognition.

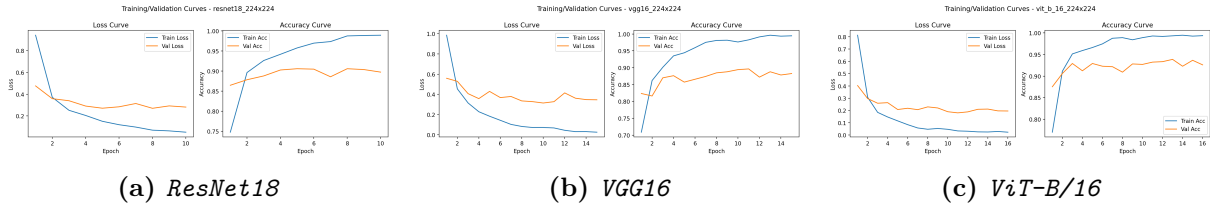


Figure 8: Training and validation curves of the classifier head for the three pretrained backbones.

The confusion matrices in Figure 9 complement the learning curves by showing how each backbone distributes its remaining errors across the 20 rice varieties. Read together with the accuracy and F1-score values, they make clear that the difference between the models is not only a matter of overall score, but also of error concentration. A stronger backbone should produce a confusion matrix with a darker and more continuous main diagonal, indicating that predictions are concentrated on the correct class, while off-diagonal activity should be sparse and localized to only a few visually similar varieties.

This is exactly the pattern observed for ViT-B/16. Its confusion matrix is the most strongly diagonal, which is consistent with its leading test accuracy of 92.18% and weighted F1-score of 92.07%. The analytical implication is that ViT does not simply improve a few easy classes while failing on the hard ones; rather, it produces a more uniformly separable feature space across the dataset. The remaining mistakes appear comparatively limited, suggesting that most classes are resolved cleanly and that confusion is restricted to only a small subset of varieties with the highest visual similarity. This supports the earlier interpretation that patch-based self-attention captures discriminative global and local structure especially well for fine-grained grain recognition.

The ResNet18 confusion matrix also shows a strong dominant diagonal, but with slightly more visible off-diagonal spread than ViT-B/16. This aligns well with its role as the most stable backbone: it generalizes very consistently, yet it still leaves a modest amount of overlap between certain classes. In other words, ResNet18 appears to make errors in a controlled and predictable way rather than failing broadly across the label space. That reading is fully consistent with the very small validation–test gap and with the idea that its embeddings are robust, even if they are not quite as separable as those produced by the transformer backbone.

By contrast, the VGG16 confusion matrix is visibly less concentrated along the diagonal, with misclassifications spread more widely across multiple class pairs. This qualitative structure helps explain why VGG16 trails the other two models despite still achieving a usable accuracy level of 86.05%. The issue is not merely that it makes more errors in total, but that the errors are less localized, indicating a weaker class organization in the extracted embedding space. Thus, the confusion matrices reinforce the same ranking already suggested by the learning

curves: ViT-B/16 offers the cleanest class separation, ResNet18 offers the most reliable and well-controlled generalization, and VGG16 remains comparatively more vulnerable to inter-class ambiguity.

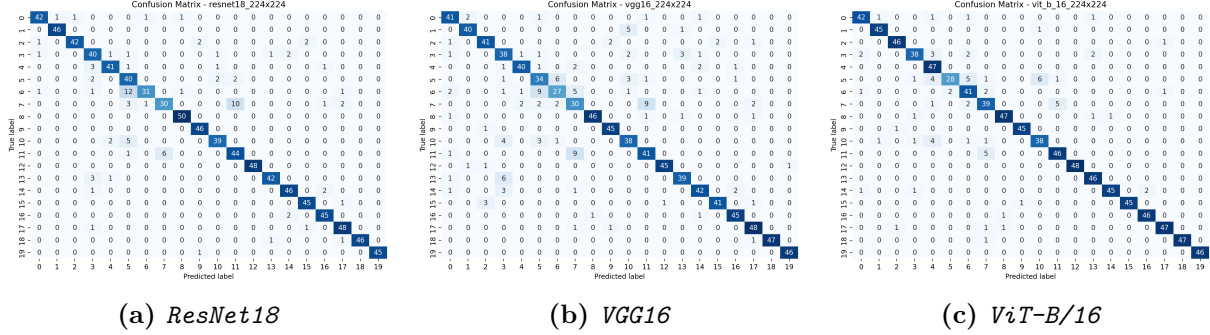


Figure 9: Test-set confusion matrices for the three pretrained backbones. A stronger diagonal concentration indicates better per-class separation, while off-diagonal mass reflects confusion between visually similar rice varieties.

Table 3: Comparison of deep-learning experiments using the custom classifier head on top of frozen pretrained embeddings.

Backbone	Best val. acc.	Test acc.	Precision	Recall	F1-score
ResNet18	90.59%	90.49%	91.09%	90.49%	90.46%
VGG16	89.64%	86.05%	86.32%	86.05%	86.05%
ViT-B/16	93.87%	92.18%	92.52%	92.18%	92.07%

5.5 Results Comparison against Traditional ML

To assess whether the learned neural classification head provides a practical advantage over conventional machine-learning models, the extracted backbone features were also evaluated with five traditional classifiers: logistic regression, linear SVM, k -nearest neighbours, random forest, and XGBoost. All five models were benchmarked for each backbone, but this subsection focuses primarily on logistic regression because it was the best-performing traditional method for all three feature. In other words, logistic regression is not the only traditional model that was tested; it is the strongest classical baseline, so it provides the fairest one-to-one comparison with the proposed deep-learning head. This comparison is analytically useful because logistic regression is a nearly linear classifier: if it already performs strongly on the extracted embeddings, then most of the class structure is already present in the feature space itself. The question is therefore not whether deep features work at all, but whether a learned nonlinear head can still refine those features into better decision boundaries.

For the vit_b_16_224x224 features, logistic regression achieved the best traditional result with validation accuracy of 91.75% and test accuracy of 91.86%, together with a macro F1-score of 91.85%. Although this is already a strong baseline, the proposed neural classifier head still performed slightly better on the same ViT features, reaching 92.18% test accuracy and 92.07% weighted F1-score. The absolute improvement is modest (0.32 percentage points in accuracy), but its meaning is important: the ViT embedding space is already so well structured that even a linear classifier can separate most classes effectively. The small gain from the neural head therefore suggests diminishing returns from adding extra classifier complexity once

the representation quality becomes sufficiently high. In practical terms, this means that the backbone is doing most of the difficult work, and the classifier stage mainly provides a modest final refinement rather than a fundamental transformation of the decision space.

For the `resnet18_224x224` features, the performance margin in favour of the neural head was larger. The best traditional model was again logistic regression, with 87.84% test accuracy and macro F1-score of 87.88%. In contrast, the proposed classifier head achieved 90.49% test accuracy and 90.46% weighted F1-score on the same feature representation. This corresponds to an improvement of about 2.64 percentage points in test accuracy. Here the larger gap is meaningful because it shows that the ResNet18 feature space is informative but not fully linearly separable. The hidden layer, batch normalization, and dropout appear to help the classifier model class boundaries that are more curved or locally complex than logistic regression can express. Thus, with ResNet18, the neural head is not merely polishing an already perfect feature space; it is contributing substantial additional discrimination that translates into better final recognition of visually similar varieties.

For the `vgg16_224x224` features, logistic regression also ranked first among the traditional methods, obtaining 85.31% test accuracy and a macro F1-score of 85.28%. The neural classification head slightly outperformed this result, reaching 86.05% test accuracy and 86.05% weighted F1-score, for an improvement of about 0.74 percentage points in accuracy. The gain is smaller than with ResNet18, but it remains consistent with the broader trend that the learned MLP-style head performs at least as well as, and usually better than, the tested classical baselines. At the same time, the limited improvement suggests that the main bottleneck for VGG16 is probably the representation itself rather than the choice of downstream classifier. If the extracted features do not encode the inter-class differences strongly enough, then replacing a linear model with a shallow nonlinear head can help only marginally. This explains why VGG16 remains behind the other backbones even when all of them are paired with the same learned head.

The broader benchmark table also highlights differences among the traditional methods themselves. Linear SVM and k -nearest neighbours generally trailed behind logistic regression, especially on the ViT and ResNet18 features, while random forest and XGBoost incurred much higher training times without producing better predictive performance. For example, on ViT features, XGBoost required substantially more fitting time than logistic regression but still produced a lower test accuracy (87.32% versus 91.86%). This is a meaningful finding because it indicates that the extracted embeddings are already arranged in a space where comparatively simple decision functions are sufficient. When tree ensembles fail to improve over logistic regression, the implication is that the problem does not primarily require more complex partitioning machinery at the classifier level; instead, success depends mainly on how well the backbone has encoded the relevant visual information.

Overall, the comparison against traditional machine learning supports two main conclusions. First, pretrained deep features are highly informative, since even simple classical models achieve strong classification accuracy; this means that representation learning, not just classifier design, is the main source of performance in this study. Second, the proposed neural classification head remains the best overall choice because it consistently matches or surpasses the strongest classical baseline for every backbone, while preserving the efficiency advantages of offline feature extraction. More specifically, the size of the improvement reveals when the head is most useful: it adds only a small benefit when the feature space is already highly separable (ViT-B/16), but a more substantial benefit when the representation still contains nonlinear class structure that

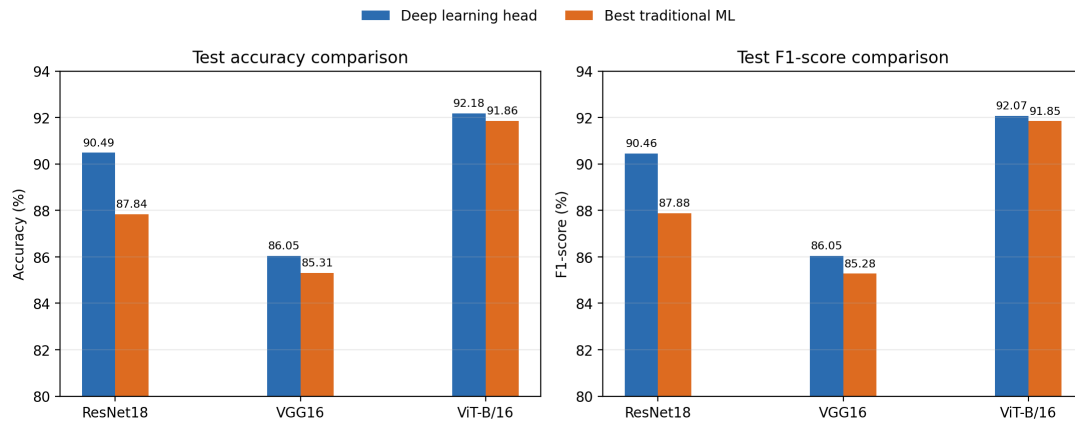


Figure 10: Comparison between the proposed deep-learning head and the best traditional baseline (logistic regression) on the same extracted features.

Table 4: Comparison between the proposed deep-learning head and the best traditional ML baseline for each backbone.

Backbone	Method	Val. acc.	Test acc.	F1-score	Fit time (s)	Δ acc.
ResNet18	Deep head	90.59%	90.49%	90.46%	–	+2.64
	Logistic regression	90.27%	87.84%	87.88%	3.76	–
VGG16	Deep head	89.64%	86.05%	86.05%	–	+0.74
	Logistic regression	86.15%	85.31%	85.28%	0.91	–
ViT-B/16	Deep head	93.87%	92.18%	92.07%	–	+0.32
	Logistic regression	91.75%	91.86%	91.85%	0.89	–

needs refinement (especially ResNet18). In practical terms, the combination of a pretrained ViT backbone with the custom classifier head gives the best final performance in this study, whereas logistic regression provides the strongest non-neural baseline and demonstrates how much of the classification problem is already solved by high-quality pretrained embeddings.

6 Conclusion

6.1 Summary of Findings

This study developed and compared a complete rice-variety image classification pipeline based on pretrained visual features, traditional machine-learning classifiers, and a lightweight deep-learning classifier head. Across all experiments, the results show that feature quality is the main driver of performance, while the choice of downstream classifier provides an additional but smaller refinement. Among the three pretrained backbones, ViT-B/16 achieved the best overall performance, reaching a best validation accuracy of 93.87%, test accuracy of 92.18%, and weighted F1-score of 92.07%. ResNet18 ranked second with 90.49% test accuracy and 90.46% weighted F1-score, while VGG16 produced the weakest but still competitive result with 86.05% test accuracy. These results indicate that transformer-based patch representations are especially effective for fine-grained rice recognition, whereas residual CNN features remain a strong and stable alternative.

The comparison against traditional machine-learning models provides a second important conclusion. Logistic regression was the strongest classical baseline for all three feature sets, achieving test accuracies of 91.86% on ViT-B/16 features, 87.84% on ResNet18 features, and 85.31% on VGG16 features. This confirms that the pretrained embeddings already organize the classes well enough for relatively simple classifiers to perform strongly. However, the proposed neural classification head consistently matched or exceeded these baselines on every backbone. The gain was largest for ResNet18 (+2.64 percentage points), moderate for VGG16 (+0.74), and smallest for ViT-B/16 (+0.32), showing that a nonlinear classifier is most helpful when the feature space is informative but not fully linearly separable. Overall, the best-performing configuration in this project is the combination of ViT-B/16 feature extraction with the custom classifier head, while logistic regression serves as an efficient and surprisingly strong non-neural baseline.

6.2 Limitations and Future Work

Despite the strong results, this study still has several limitations. First, the deep-learning stage relies on frozen pretrained embeddings rather than end-to-end fine-tuning. This design is computationally efficient and well suited to the current dataset size, but it may limit the model's ability to adapt fully to subtle domain-specific patterns in rice-grain imagery. Second, the comparison is restricted to three pretrained backbones and a shallow classifier head, so the design space of modern architectures is not explored exhaustively. Third, the evaluation is based on a single dataset with 20 rice varieties; therefore, the reported performance may not fully reflect real-world variation in lighting, capture devices, backgrounds, or unseen grain distributions. Finally, the project emphasizes predictive accuracy more than deployment concerns such as inference latency, memory usage, model compression, and robustness under distribution shift.

Future work can extend this project in several meaningful directions. A natural next step is to fine-tune the best-performing backbone, especially ViT-B/16 and ResNet18, to determine whether end-to-end adaptation yields further gains beyond feature-based transfer learning. It would also be valuable to evaluate additional architectures and training strategies, such as stronger augmentation policies, metric-learning objectives, self-supervised pretraining, or attention-based hybrid CNN–Transformer models. From the machine-learning perspective,



future studies could analyze class-wise errors in more detail and incorporate explainability tools to identify which grain characteristics most strongly influence predictions. Finally, a practical extension would be to test the pipeline on more diverse field data and optimize the best model for real deployment in agricultural support systems, where both accuracy and computational efficiency are important.

References

- [1] M. S. Uddin, *An image dataset of rice varieties*, version 3, Accessed: 2025, Dec. 2023. DOI: 10.17632/3mn9843tz2.3 [Online]. Available: <https://data.mendeley.com/datasets/3mn9843tz2/3>
- [2] S. J. Pan and Q. Yang, “A survey on transfer learning,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010. DOI: 10.1109/TKDE.2009.191 [Online]. Available: <https://doi.org/10.1109/TKDE.2009.191>
- [3] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90 [Online]. Available: <https://doi.org/10.1109/CVPR.2016.90>
- [4] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *International Conference on Learning Representations (ICLR)*, 2015, arXiv:1409.1556. [Online]. Available: <https://arxiv.org/abs/1409.1556>
- [5] A. Dosovitskiy et al., “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: <https://openreview.net/forum?id=YicbFdNTTy>